

CI1

Réalisé par Meriam Boukraa

Exercice 1:

1.a. Connexion via SSH:

```
meria@Meriam: ~  
meria@Meriam: /mnt/c/Users/  
(base) meria@Meriam:~$ ssh tsp-client  
mboukraa@ssh3.imtbs-tsp.eu's password:  
The authenticity of host '157.159.104.128 (<no hostip for proxy command>)' c  
an't be established.  
ED25519 key fingerprint is SHA256:dtb8QlVFBt8fswz7U8sLeuwmDZbiYlRd0vrkqn0Q80  
k.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '157.159.104.128' (ED25519) to the list of known  
hosts.  
mboukraa@157.159.104.128's password:
```

<pre>new interface is .eu/albator/ ule Slurm jobs. Please be sure to User Guide, under feedback on the -sudparis.eu</pre>	<pre>*** NOTICE *** Summer maintenance on the cluster is complete. A available online: https://de.telecom-sudparis New scripts are now available to help you sched This year, many courses will be using the cluster. schedule your jobs accordingly. Refer to the online the Best Practices section. Students and staff, we welcome your comments and cluster's performance. arthur.jovart@telecom</pre>
---	--

```
meria@Meriam: ~  
meria@Meriam: /mnt/c/Users/  
##### DE TSP #####  
#####  
Welcome to the Albator HPC Cluster  
Operated by the Direction de l'Enseignement – Télécom SudParis  
Useful commands:  
  squeue          List running and pending jobs  
  squeue -u $USER List your own jobs  
  sbatch script.sh Submit a job  
  scancel <jobid> Cancel a job  
  sinfo           View cluster node status  
  sacct           View your job history  
Web panel: https://de.telecom-sudparis.eu/albator/  
Support:   arthur.jovart@telecom-sudparis.eu  
This cluster is reserved for teaching purposes only.  
---  
Bienvenue sur le cluster HPC Albator  
Opéré par la Direction de l'Enseignement – Télécom SudParis  
Commandes utiles :  
  squeue          Lister les jobs en cours et en attente  
  squeue -u $USER Lister vos propres jobs  
  sbatch script.sh Soumettre un job  
  scancel <jobid> Annuler un job  
  sinfo           Afficher l'état des nœuds du cluster  
  sacct           Afficher l'historique de vos jobs  
Interface web : https://de.telecom-sudparis.eu/albator/  
Support :      arthur.jovart@telecom-sudparis.eu  
Ce cluster est réservé à l'enseignement.  
#####  
#####
```

1.c.

Avec la commande `nvidia-smi`, on peut voir le modèle exact du GPU qui nous a été alloué. C'est NVIDIA L4:

```

meria@Meriam: ~
meria@Meriam: /mnt/c/Users/

[controller] mboukraa@arcadia-slurm-controller:~
$ nvidia-smi
Command 'nvidia-smi' not found, but can be installed with:
apt install nvidia-utils-580 # version 580.159.03-0ubuntu0.26.04.1,
or
apt install nvidia-utils-580-server # version 580.159.03-0ubuntu0.26.04.1
apt install nvidia-utils-595 # version 595.71.05-0ubuntu0.26.04.1
apt install nvidia-utils-595-server # version 595.71.05-0ubuntu0.26.04.1
Ask your administrator to install one of them.
X127 [controller] mboukraa@arcadia-slurm-controller:~
$ srun --partition=gpu --gres=gpu:1 --time=01:00:00 --cpus-per-task=1 --mem=
8G --pty bash
[compute] [job 1526] mboukraa@starfighter-slurm-node-06-1:~
$ nvidia-smi
Wed Sep 16 10:06:44 2026
+-----+
+-----+
| NVIDIA-SMI 595.84 Driver Version: 595.84 CUDA Vers
ion: 13.2 |
+-----+
+-----+
| GPU Name Persistence-M | Bus-Id Disp.A | Volatil
e Uncorr. ECC |
| Fan Temp Perf Pwr:Usage/Cap | Memory-Usage | GPU-Uti
l Compute M. |
| MIG M. |
+-----+
|====+
=====|
| 0 NVIDIA L4 Off | 00000000:06:10.0 Off |
| 0 |
| N/A 34C P8 15W / 72W | 3MiB / 23034MiB | 0%
| Default |
| N/A |
+-----+
+-----+
+-----+
+-----+
| Processes:

```

1.d.

Le jobID de mon job interactif est 1534:

```

[controller] mboukraa@arcadia-slurm-controller:~
$ squeue -u $USER
JOBID PARTITION NAME USER ST TIME NODES NODELIST
(REASON)
1534 gpu bash mboukraa R 53:26 1 starfigh
ter-slurm-node-02-1
[controller] mboukraa@arcadia-slurm-controller:~
$

```

La commande exacte pour annuler ce job est scancel 1534:

```
[controller] mboukraa@arcadia-slurm-controller:~  
$ scancel 1534  
[controller] mboukraa@arcadia-slurm-controller:~  
$ |
```

On peut ensuite vérifier l'annulation de ce job:

```
[controller] mboukraa@arcadia-slurm-controller:~  
$ squeue -u $USER  
          JOBID PARTITION      NAME      USER ST       TIME  NODES NODELIST  
(REASON)  
[controller] mboukraa@arcadia-slurm-controller:~  
$ |
```

1.e.

Le contenu du fichier [hello.sh](#): (On a après rendu le script exécutable et on a soumis le Job)

```
[controller] mboukraa@arcadia-slurm-controller:~  
$ nano hello.sh  
[controller] mboukraa@arcadia-slurm-controller:~  
$ cat hello.sh  
#!/bin/bash  
#SBATCH --partition=gpu  
#SBATCH -t 1:00:00  
#SBATCH --gres=gpu:1  
#SBATCH --cpus-per-task=1  
#SBATCH --mem=8G  
#SBATCH -J hello-slurm  
#SBATCH -o logs/%x-%j.out  
#SBATCH -e logs/%x-%j.err  
  
set -euo pipefail  
mkdir -p logs  
  
echo "Job $SLURM_JOB_ID on $SLURM_NODELIST"  
nvidia-smi || echo "nvidia-smi indisponible"  
  
echo "Bonjour depuis SLURM !"  
[controller] mboukraa@arcadia-slurm-controller:~  
$ chmod +x hello.sh  
[controller] mboukraa@arcadia-slurm-controller:~  
$ ls -l hello.sh  
-rwxr-xr-x mboukraa 1617 348 B Wed Sep 16 12:06:50 2026 ♦ hello.sh  
[controller] mboukraa@arcadia-slurm-controller:~  
$ sbatch hello.sh  
Submitted batch job 1537  
[controller] mboukraa@arcadia-slurm-controller:~
```

Après on regarde le contenu du fichier de log. Le nom exact du fichier de log généré est: hello-slurm-1537.out

```
meria@Meriam: ~
meria@Meriam: /mnt/c/Users
$ ls logs
hello-slurm-1537.err  hello-slurm-3825.err
hello-slurm-1537.out  hello-slurm-3825.out
[controller] mboukraa@arcadia-slurm-controller:~
$ cat logs/hello-slurm-1537.out
Job 1537 on starfighter-slurm-node-03-1
Wed Sep 16 12:10:22 2026
-----
+-----+
| NVIDIA-SMI 595.84                Driver Version: 595.84          CUDA Vers
ion: 13.2                |
+-----+-----+-----+-----+
+-----+
| GPU   Name                     Persistence-M | Bus-Id        Disp.A | Volatil
e Uncorr. ECC |
| Fan   Temp   Perf              Pwr:Usage/Cap |           Memory-Usage | GPU-Uti
l   Compute M. |
|               MIG M. |
+-----+-----+-----+-----+
| 0     NVIDIA L4                Off | 00000000:06:10.0 Off |
| 0     |
| N/A   32C    P8                15W / 72W | 0MiB / 23034MiB | 0%
|      Default |
|               |
|      N/A |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
| GPU   GI    CI               PID   Type   Process name
| GPU Memory |
|      ID    ID
|      Usage   |
+-----+-----+-----+-----+
| No running processes found
```

1.f.

ReqMem correspond à la quantité de mémoire demandée à SLURM pour le job. Dans mon cas, j'ai demandé 8 Go. MaxRSS correspond à la quantité maximale de mémoire réellement utilisée par le job pendant son exécution. Ainsi, ReqMem représente la mémoire réservée/demandée, tandis que MaxRSS permet de voir la mémoire effectivement utilisée au maximum.

```
[controller] mboukraa@arcadia-slurm-controller:~
$ sacct -u $USER
JobID          JobName      Partition    Account    AllocCPUS      State  ExitCode
-----
1526            bash        gpu         pro_8605         1    TIMEOUT      0:0
1526.extern     extern              pro_8605         1    COMPLETED    0:0
1526.0          bash              pro_8605         1    CANCELLED
1534            bash        gpu         pro_8605         1    RUNNING      0:0
1534.extern     extern              pro_8605         1    RUNNING      0:0
1534.0          bash              pro_8605         1    RUNNING      0:0
1537            hello-slu+   gpu         pro_8605         1    COMPLETED    0:0
1537.batch      batch              pro_8605         1    COMPLETED    0:0
1537.extern     extern              pro_8605         1    COMPLETED    0:0

[controller] mboukraa@arcadia-slurm-controller:~
$ sacct -j 1537 --format=JobID,State,Elapsed,MaxRSS,ReqMem,ReqCPUS
JobID          State      Elapsed      MaxRSS      ReqMem      ReqCPUS
-----
1537            COMPLETED  00:00:01              8G          1
1537.batch      COMPLETED  00:00:01      17988K          1
1537.extern     COMPLETED  00:00:02          1
[controller] mboukraa@arcadia-slurm-controller:~
```

1.g.

Depuis ma machine vers le cluster:

Comme je n'ai pas mnist.zip, j'ai créé un petit fichier de test: test.txt. J'ai aussi créé le dossier data. ensuite j'ai fait le transfert par: scp test.txt tsp-client:~/data/

```
meria@Meriam: ~ × meria@Meriam: / × meria@Meriam: , × + ▾ - □  
[controller] mboukraa@arcadia-slurm-controller:~  
$ ls ~  
♦ hello.sh ♦ logs  
[controller] mboukraa@arcadia-slurm-controller:~  
$ mkdir -p ~/data  
[controller] mboukraa@arcadia-slurm-controller:~  
$ ls ~  
♦ data ♦ hello.sh ♦ logs  
[controller] mboukraa@arcadia-slurm-controller:~  
$ exit  
exit  
Connection to 157.159.104.128 closed.  
(base) meria@Meriam:/mnt/c/Users/meria$ scp test.txt tsp-client:~/data/  
mboukraa@ssh3.imtbs-tsp.eu's password:  
mboukraa@157.159.104.128's password:  
test.txt 100% 19 0.1KB/s 00:00
```

Pour vérifier que le fichier est bien arrivé:

```
[controller] mboukraa@arcadia-slurm-controller:~  
$ cat ~/data/test.txt  
Test transfert TP1  
[controller] mboukraa@arcadia-slurm-controller:~  
$ |
```

Depuis le cluster vers ma machine:

J'ai créé le fichier sur le cluster:

```
[controller] mboukraa@arcadia-slurm-controller:~  
$ mkdir -p ~/results  
[controller] mboukraa@arcadia-slurm-controller:~  
$ echo "Résultat du test TP1" > ~/results/output.log  
[controller] mboukraa@arcadia-slurm-controller:~  
$ cat ~/results/output.log  
Résultat du test TP1  
[controller] mboukraa@arcadia-slurm-controller:~  
$ exit  
exit  
Connection to 157.159.104.128 closed.  
(base) meria@Meriam:/mnt/c/Users/meria$ |
```

Après j'ai téléchargé le fichier vers ma machine et j'ai vérifié qu'il est bien arrivé:

```
(base) meria@Meriam:/mnt/c/Users/meria$ scp tsp-client:~/results/output.log
./output.log
mboukraa@ssh3.imtbs-tsp.eu's password:
mboukraa@157.159.104.128's password:
output.log                                100% 22      0.1KB/s   00:00
(base) meria@Meriam:/mnt/c/Users/meria$ cat output.log
Résultat du test TP1
(base) meria@Meriam:/mnt/c/Users/meria$ |
```

Synchroniser un dossier:

J'ai commencé par créer quelques fichiers localement:

```
(base) meria@Meriam:/mnt/c/Users/meria$ mkdir -p results
(base) meria@Meriam:/mnt/c/Users/meria$ echo "Premier fichier" > results/fic
hier1.txt
echo "Deuxième fichier" > results/fichier2.txt
(base) meria@Meriam:/mnt/c/Users/meria$ ls results/
fichier1.txt  fichier2.txt
(base) meria@Meriam:/mnt/c/Users/meria$ |
```

Ensuite j'ai lancé rsync:

```
(base) meria@Meriam:/mnt/c/Users/meria$ rsync -avhP ./results/ tsp-client:~/
backup-results/
mboukraa@ssh3.imtbs-tsp.eu's password:
mboukraa@157.159.104.128's password:
sending incremental file list
created directory /mnt/hdd/homes/mboukraa/backup-results
./
fichier1.txt
      16 100%    0.00kB/s   0:00:00 (xfr#1, to-chk=1/3)
fichier2.txt
      18 100%    5.86kB/s   0:00:00 (xfr#2, to-chk=0/3)

sent 226 bytes  received 118 bytes  20.85 bytes/sec
total size is 34  speedup is 0.10
(base) meria@Meriam:/mnt/c/Users/meria$ |
```

Les fichiers sont bien transférés:

```
[controller] mboukraa@arcadia-slurm-controller:~
$ ls ~/backup-results/
♦ fichier1.txt ♦ fichier2.txt
[controller] mboukraa@arcadia-slurm-controller:~
$ cat ~/backup-results/fichier1.txt
Premier fichier
[controller] mboukraa@arcadia-slurm-controller:~
$ cat ~/backup-results/fichier2.txt
Deuxième fichier
[controller] mboukraa@arcadia-slurm-controller:~
$ |
```


Exercice 2:

2.a. Création de Miniforge:

J'ai réservé un noeud de calcul pour que les installations se font dessus:

```
[controller] mboukraa@arcadia-slurm-controller:~  
$ srun --partition=gpu --gres=gpu:1 --time=01:00:00 --cpus-per-task=1 --mem=8G --pty bash  
[compute] [job 1555] mboukraa@starfighter-slurm-node-02-1:~  
$ |
```

Installation de Miniforge:

```
[compute] [job 1555] mboukraa@starfighter-slurm-node-02-1:~  
$ curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-$(uname)-$(uname -m).sh"  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
nt                                     Dload  Upload  Total  Spent  Left  Speed  
0          0  0      0  0      0     0      0      0      0      0      0      0  
0          0  0      0  0      0     0      0      0      0      0      0      0  
0          0  0      0  0      0     0      0      0      0      0      0      0  
0          0  0      0  0      0     0      0      0      0      0      0      0  
0          0  0      0  0      0     0      0      0      0      0      0      0  
26 118.7M  26 31.99M  0      0  23.41M      0  00:05  00:01  00:04 31.99  
53 118.7M  53 63.35M  0      0  26.77M      0  00:04  00:02  00:02 31.69  
76 118.7M  76 91.27M  0      0  27.12M      0  00:04  00:03  00:01 30.44  
100 118.7M 100 118.7M  0      0  27.26M      0  00:04  00:04      0 30.44  
100 118.7M 100 118.7M  0      0  27.26M      0  00:04  00:04      0 30.44  
100 118.7M 100 118.7M  0      0  27.26M      0  00:04  00:04      0 30.44  
M  
[compute] [job 1555] mboukraa@starfighter-slurm-node-02-1:~  
$ ls  
❖ backup-results  ❖ hello.sh  ❖ Miniforge3-Linux-x86_64.sh  
❖ data            ❖ logs      ❖ results  
[compute] [job 1555] mboukraa@starfighter-slurm-node-02-1:~  
$ |
```

```
[compute] [job 1555] mboukraa@starfighter-slurm-node-02-1:~
$ bash Miniforge3-${uname}-${uname -m}.sh

Welcome to Miniforge3 26.7.2-0

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>>
Miniforge installer code uses BSD-3-Clause license as stated below.

Binary packages that come with it have their own licensing terms
and by installing miniforge you agree to the licensing terms of individual
packages as well. They include different OSI-approved licenses including
the GNU General Public License and can be found in pkgs/<pkg-name>/info/lice
nses
folders.

Miniforge installer comes with a bootstrapping executable that is used
when installing miniforge and is deleted after miniforge is installed.
The bootstrapping executable uses micromamba, cli11, cpp-fileSYSTEM,

Thank you for installing Miniforge3!
[compute] [job 1555] mboukraa@starfighter-slurm-node-02-1:~
$ |
```

Et j'ai bien vérifié que Mamba fonctionne:

```
[compute] [job 1559] mboukraa@starfighter-slurm-node-01-1:~
$ mamba --version
2.9.0
[compute] [job 1559] mboukraa@starfighter-slurm-node-01-1:~
$ |
```

Création et activation de l'environnement deeplearning:

```
Linking wheel-0.48.0-pyhd8ed1ab_0
Linking pip-26.2.1-pyh8b19718_0

Transaction finished

To activate this environment, use:

    mamba activate deeplearning

Or to execute a single command in this environment, use:

    mamba run -n deeplearning mycommand

[compute] [job 1559] mboukraa@starfighter-slurm-node-01-1:~
$ mamba activate deeplearning
[compute] [job 1559] mboukraa@starfighter-slurm-node-01-1:~
$ |
```

Pour vérifier la version de Python et le chemin du binaire utilisé dans l'environnement actif, j'ai utilisé les commandes :

```
python --version  
which python
```

Résultat :

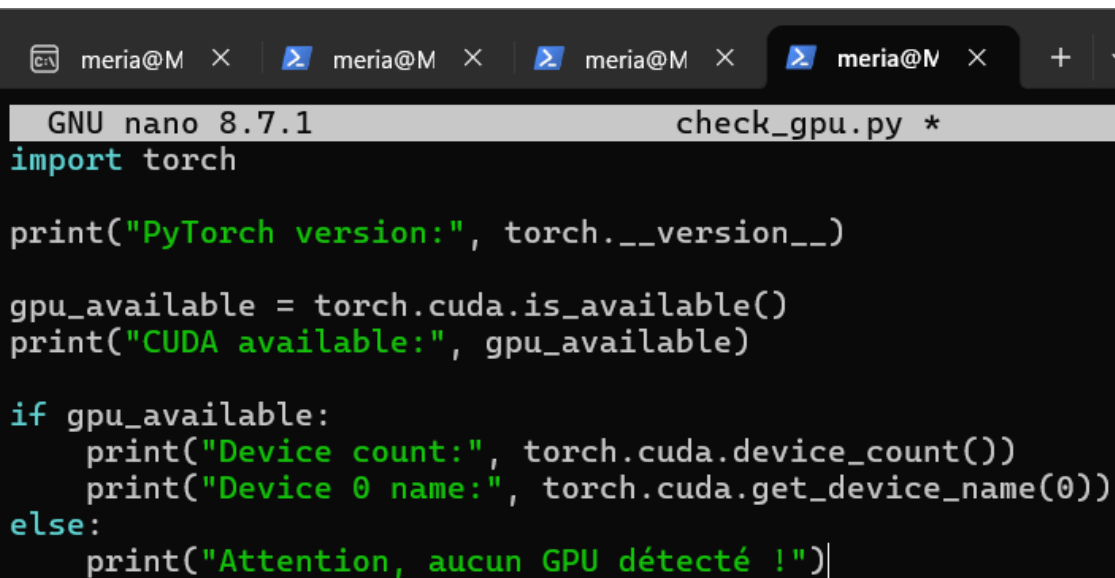
Python 3.10.21

/mnt/hdd/homes/mboukraa/miniforge3/envs/deeplearning/bin/python

```
$ python --version  
Python 3.10.21  
[compute] [job 1559] mboukraa@starfighter-slurm-node-01-1:~  
$ which python  
/mnt/hdd/homes/mboukraa/miniforge3/envs/deeplearning/bin/python  
[compute] [job 1559] mboukraa@starfighter-slurm-node-01-1:~  
$
```

2.d. Vérification de l'installation PyTorch + CUDA:

J'ai commencé par remplir le fichier check_gpu.py:



```
GNU nano 8.7.1 check_gpu.py *  
import torch  
  
print("PyTorch version:", torch.__version__)  
  
gpu_available = torch.cuda.is_available()  
print("CUDA available:", gpu_available)  
  
if gpu_available:  
    print("Device count:", torch.cuda.device_count())  
    print("Device 0 name:", torch.cuda.get_device_name(0))  
else:  
    print("Attention, aucun GPU détecté !")|
```

Après j'ai exécuté le script:

```
[compute] [job 1728] mboukraa@arcadia-slurm-node-2:~
$ python check_gpu.py
PyTorch version: 2.14.0+cu130
CUDA available: True
Device count: 1
Device 0 name: NVIDIA H100 NVL
[compute] [job 1728] mboukraa@arcadia-slurm-node-2:~
$
```

2.e. J'ai rendu l'environnement reproductible et il a bien été ajouté dans le dossier TP1 de mon dépôt Git et a été commit:

```
[compute] [job 1562] mboukraa@starfighter-slurm-node-01-1:~
$ mamba env export --from-history -n deeplearning > environment.yml
[compute] [job 1562] mboukraa@starfighter-slurm-node-01-1:~
$ ls -l environment.yml
-rw-r--r-- mboukraa 1617 240 B Wed Sep 16 16:04:00 2026 ♦ environment.yml
[compute] [job 1562] mboukraa@starfighter-slurm-node-01-1:~
$ cat environment.yml
name: deeplearning
channels:
  - conda-forge
  - pytorch
dependencies:
  - pip
  - python=3.10
  - pytorch
  - pytorch-cuda=12.1
  - tensorboard
  - torchaudio
  - torchvision

prefix: "/mnt/hdd/homes/mboukraa/miniforge3/envs/deeplearning"
[compute] [job 1562] mboukraa@starfighter-slurm-node-01-1:~
```

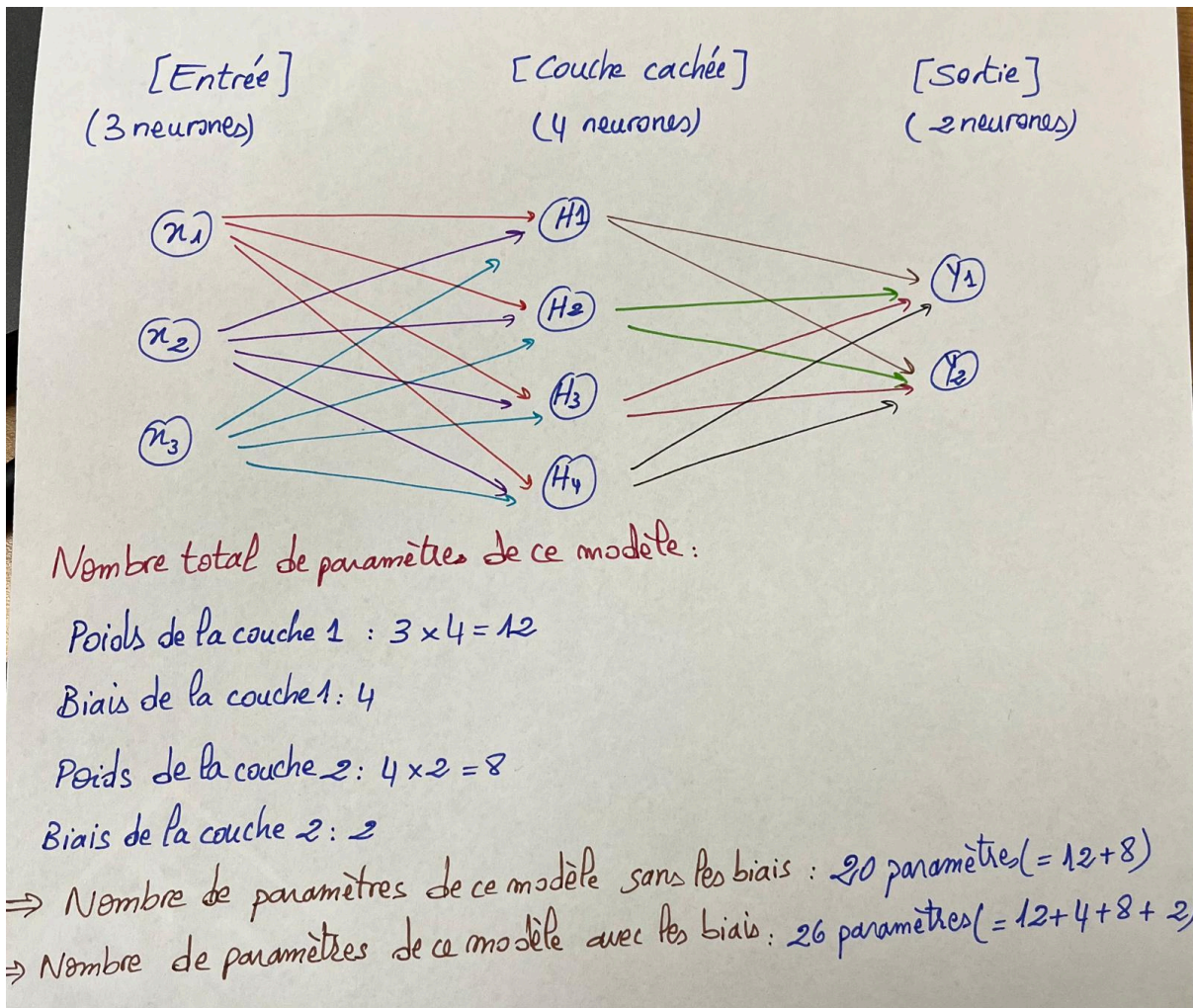
MeriamBkrr Ajout des fichiers du TP1 7b6ee84 · 13 minutes ago History		
Name	Last commit message	Last commit date
..		
check_gpu.py	Ajout des fichiers du TP1	13 minutes ago
environment.yml	Ajout des fichiers du TP1	13 minutes ago
hello.sh	Ajout des fichiers du TP1	13 minutes ago

2.f. Finalement, j'ai bien vérifié que TensorBoard a été bien installé. La commande est:
tensorboard --version

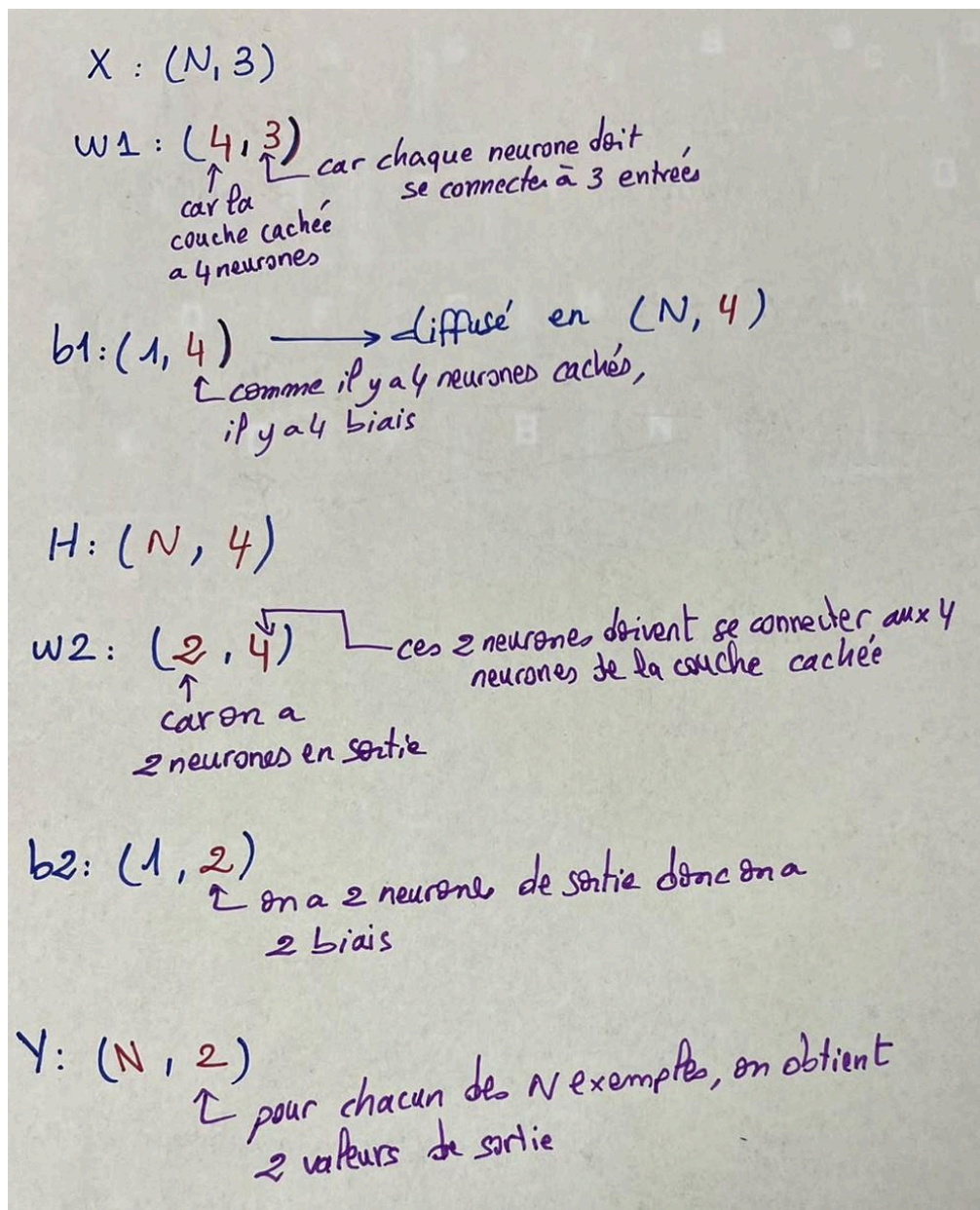
```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP1 (main)
$ tensorboard --version
/mnt/hdd/homes/mboukraa/miniforge3/envs/deeplearning/lib/python3.10/site-packages/tensorboard/default.py:30: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.
  import pkg_resources
TensorFlow installation not found - running with reduced feature set.
2.20.0
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP1 (main)
$ |
```

Exercice 3:

3.a. Schéma du perceptron multicouche (MLP):



3.b.



3.c.

1.

$$f(x, y, z) = (x/y) + z$$

Grapher le calcul de cette fonction:

$$x \xrightarrow{\quad} q = \frac{x}{y}$$

$$\downarrow$$

$$f = q + z$$

Diagram illustrating the calculation of the function $f(x, y, z) = (x/y) + z$. It shows the flow from x to $q = x/y$, and then from q and z to $f = q + z$.

2.

Le forward pass:

Etape 1: Calculer q :

$$q = \frac{x}{y} = \frac{2}{4} = 0,5$$

Etape 2: Calculer f :

$$f = q + z = 0,5 + 0 = 0,5$$

3.

Backpropagation:

On veut trouver $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$

* On a $f = q + z$ donc $\frac{\partial f}{\partial q} = 1$ et $\boxed{\frac{\partial f}{\partial z} = 1}$

* On a $q = \frac{x}{y}$

Donc $\frac{\partial q}{\partial x} = \frac{1}{y}$. Or $y = 4$ donc $\frac{\partial q}{\partial x} = \frac{1}{4} = 0,25$

Aussi $\frac{\partial q}{\partial y} = \frac{-x}{y^2}$. Or $x = 2$ et $y = 4$ donc $\frac{\partial q}{\partial y} = \frac{-2}{4^2} = \frac{-2}{16} = -0,125$

$\Rightarrow$ * $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \times \frac{\partial q}{\partial x} = 1 \times 0,25 \Rightarrow \boxed{\frac{\partial f}{\partial x} = 0,25}$

* $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \times \frac{\partial q}{\partial y} = 1 \times (-0,125) \Rightarrow \boxed{\frac{\partial f}{\partial y} = -0,125}$

3.d.

On a : $\frac{\partial f}{\partial x} = 0,25$, $\frac{\partial f}{\partial y} = -0,125$, $\frac{\partial f}{\partial z} = 1$

Avec la descente de gradient on a :

$$x' = x - \eta \frac{\partial f}{\partial x} \quad , \quad y' = y - \eta \frac{\partial f}{\partial y} \quad , \quad z' = z - \eta \frac{\partial f}{\partial z}$$

avec $\eta = 1$

$$\Rightarrow x' = 2 - 1 \times 0,25 \Rightarrow \boxed{x' = 1,75}$$

$$y' = 4 - 1 \times (-0,125) \Rightarrow \boxed{y' = 4,125}$$

$$z' = 0 - 1 \times 1 \Rightarrow \boxed{z' = -1}$$

$$\Rightarrow f' = \frac{1,75}{4,125} + (-1) \Rightarrow \boxed{f' = -0,5758}$$

Où, la valeur de f' est inférieure à f

$\Rightarrow$ La valeur de la fonction a diminué comme attendu

3.e.

* La règle de la chaîne permet de calculer le gradient de la perte par rapport aux paramètres en tenant compte de toutes les transformations successives réalisées par les différentes couches du réseau.

* Les mini-batch offrent un compromis entre la mise à jour sur un seul exemple, qui produit des gradients très bruités, et la mise à jour sur l'ensemble des données, qui est coûteuse en temps et en mémoire.

3.f.

Tâche	Fonction finale (Sortie)	Fonction de perte
Classification binaire	Sigmoïde	Binary Cross-Entropy
Classification multi	Soft Max	Cross-Entropy
Régression pure	Identité (aucune)	MSE

Exercice 4:

4.a.

Voici le code complété de l'étape 1:

```
meria@Meriam: /mnt/c/Users  ×  +  ∨
GNU nano 8.7.1 train.py *
import os
import torch
import torchvision
from torchvision import transforms, datasets

# Normalisation "classique" pour CIFAR-10
CIFAR10_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR10_STD = (0.2023, 0.1994, 0.2010)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),
])

# Chargement des données
trainset = datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

testset = datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

# Ne pas monopoliser les CPUs sur Slurm
def get_num_workers(default=2, cap=4):
    try:
        n = int(os.getenv("SLURM_CPUS_PER_TASK", default))
    except Exception:
        n = default
    return max(0, min(cap, n))
```

L'argument **batch_size** indique le nombre d'exemples traités simultanément avant une mise à jour des poids, ici, `batch_size=32` signifie que les images sont traitées par mini-batches de 32. L'argument **shuffle** permet de mélanger les données: il est activé (`True`) pour

l'entraînement afin de varier l'ordre des exemples à chaque époque, mais désactivé (False) pour le test car aucun apprentissage n'est effectué et l'ordre des données n'a pas d'impact sur l'accuracy.

4.b.

Voici la partie du code à ajouter complète:

```
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        # Image 32x32 avec 3 canaux (RGB) -> aplatie
        self.fc1 = nn.Linear(3072, 128) # couche cachée
        self.fc2 = nn.Linear(128, 10)   # couche de sortie (logits)

    def forward(self, x):
        # Aplatir de manière robuste en préservant la dimension batch (dim 0)
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x) # NE PAS appliquer Softmax ici
        return x
```

Réponses aux questions:

1. Pourquoi utilise-t-on `torch.flatten(x, 1)` dans la méthode `forward` ?

Les images CIFAR-10 ont une dimension de (3, 32, 32), correspondant aux 3 canaux RGB et aux dimensions 32×32 de l'image. Or, une couche linéaire `nn.Linear` attend en entrée un vecteur de caractéristiques. `torch.flatten(x, 1)` permet donc d'aplatir chaque image en un vecteur de $3 \times 32 \times 32 = 3072$ valeurs, tout en conservant la dimension du batch. Ainsi, un batch de 32 images passe de (32, 3, 32, 32) à (32, 3072).

2. Pourquoi ne faut-il pas ajouter de Softmax à la fin du réseau avec `nn.CrossEntropyLoss` ?

Il ne faut pas appliquer Softmax avant `nn.CrossEntropyLoss` car cette fonction de perte attend directement les logits bruts produits par le réseau. `CrossEntropyLoss` réalise déjà en interne les opérations nécessaires liées au calcul du Softmax, de manière numériquement stable. Ajouter un Softmax avant la fonction de perte serait donc inutile et pourrait entraîner une perte de stabilité numérique.

4.c.

Voici une partie du code de cette étape après avoir complété les zones manquantes:

```

for inputs, labels in trainloader:
    inputs = inputs.to(device, non_blocking=True)
    labels = labels.to(device, non_blocking=True)

    # 1. Réinitialiser les gradients
    optimizer.zero_grad(set_to_none=True)

    # 2. Passe avant (forward)
    outputs = model(inputs)

    # 3. Calcul de la perte
    loss = criterion(outputs, labels)

    # 4. Passe arrière (Rétropropagation)
    loss.backward()

    # 5. Mise à jour des poids
    optimizer.step()

    # Statistiques
    running_loss += loss.item() * inputs.size(0)
    preds = outputs.argmax(dim=1)
    running_correct += (preds == labels).sum().item()
    running_total += labels.size(0)

epoch_loss = running_loss / running_total
epoch_acc = running_correct / running_total

print(
    f"Epoch {epoch+1:02d} | "
    f"loss={epoch_loss:.4f} | "
    f"acc={epoch_acc:.4f}"
)

```

On voit bien que l'environnement GPU est bien disponible:

```

$ mamba activate deeplearning
[compute] [job 2263] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP1 (main)
$ python -c "import torch; print('PyTorch:', torch.__version__); print('CUDA:', torch.version.cuda); print('GPU disponible:', torch.cuda.is_available()); print('GPU:', torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'AUCUN')"
PyTorch: 2.14.0+cu130
CUDA: 13.0
GPU disponible: True
GPU: NVIDIA L4

```

Et on voit bien que mon réseau de neurones s'est bien entraîné sur le GPU NVIDIA L4:

```
[compute] [job 2263] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP1 (main)
$ python train.py
Using device: cuda
Epoch 01 | loss=2.0883 | acc=0.3304
Epoch 02 | loss=2.1281 | acc=0.3560
Epoch 03 | loss=2.1288 | acc=0.3650
Epoch 04 | loss=2.0725 | acc=0.3836
Epoch 05 | loss=2.0766 | acc=0.3862
Epoch 06 | loss=2.0587 | acc=0.3939
Epoch 07 | loss=2.0087 | acc=0.4081
Epoch 08 | loss=2.0123 | acc=0.4098
Epoch 09 | loss=1.9616 | acc=0.4203
Epoch 10 | loss=1.9358 | acc=0.4262
[compute] [job 2263] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP1 (main)
$ |
```

Après 10 époques d'entraînement sur le GPU NVIDIA L4, l'accuracy d'entraînement atteint 42,62 %, contre 33,04 % lors de la première époque. La diminution globale de la fonction de perte et l'augmentation de l'accuracy montrent que le modèle apprend progressivement à partir des données d'entraînement.

Réponse à la question:

`optimizer.zero_grad()` → efface les gradients précédemment calculés avant de calculer ceux du nouveau batch.

`loss.backward()` → calcule les gradients de la fonction de perte par rapport aux paramètres du modèle grâce à la rétropropagation.

4.d.

Voici le code à ajouter pour cette étape:

```
model.eval() # Mode évaluation
classes = trainset.classes

total = 0
correct = 0

with torch.no_grad():
    for images, labels in testloader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

acc = correct / total
print(f"Test accuracy: {acc:.3f}")
```

Et on voit bien maintenant que notre modèle a fait l'évaluation sur le test set (Test accuracy: 0.389):

```
[compute] [job 2263] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP1 (main)
$ python train.py
Using device: cuda
Epoch 01 | loss=2.0883 | acc=0.3304
Epoch 02 | loss=2.1281 | acc=0.3560
Epoch 03 | loss=2.1288 | acc=0.3650
Epoch 04 | loss=2.0725 | acc=0.3836
Epoch 05 | loss=2.0766 | acc=0.3862
Epoch 06 | loss=2.0587 | acc=0.3939
Epoch 07 | loss=2.0087 | acc=0.4081
Epoch 08 | loss=2.0123 | acc=0.4098
Epoch 09 | loss=1.9616 | acc=0.4203
Epoch 10 | loss=1.9358 | acc=0.4262
Test accuracy: 0.389
[compute] [job 2263] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP1 (main)
$ |
```

Réponses aux questions:

1. Le bloc `with torch.no_grad()` est utilisé lors de l'évaluation car il n'est pas nécessaire de calculer les gradients lorsque le modèle ne fait que des prédictions. Cela évite de construire le graphe de calcul nécessaire à la rétropropagation et permet ainsi de réduire l'utilisation de la mémoire GPU et les calculs, ce qui rend l'évaluation plus efficace.
2. CIFAR-10 comporte 10 classes. Si les classes étaient prédites de manière totalement aléatoire, la probabilité de choisir la bonne classe serait de 1/10, soit une accuracy d'environ 10 %.

4.e.

Le bout de code à ajouter pour faire la sauvegarde:

```
# Sauvegarde des poids
torch.save(model.state_dict(), "mlp_model.pth")
```

Et après avoir lancé le `train.py` encore une fois, on vérifie l'existence de `mlp_model.pth`:

```
[compute] [job 2263] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP1 (main)
$ ls
❖ check_gpu.py  ❖ environment.yml  ❖ logs  ❖ train.py
❖ data          ❖ hello.sh          ❖ mlp_model.pth
[compute] [job 2263] mboukraa@starfighter-slurm-node-01-1:~/TP-HPC/TP1 (main)
```

Exercice 5:

5.a.

Voici le code complet de cette étape:


```
meria@Meriam: /mnt/c/Users  ×  +  ▾
GNU nano 8.7.1                train_tb.py *
run_name = f"{hparams['model']}/bs{hparams['batch_size']}_lr{hparams['lr']}">

logdir = os.path.join("runs", run_name)

print("Logdir:", logdir)

# Instanciation du SummaryWriter
writer = SummaryWriter(log_dir=logdir)

# 2. Séparation du jeu de données (on suppose 'trainset' déjà chargé comme >
N = len(trainset)

val_size = int(0.1 * N)
train_size = N - val_size

# Utilisation de random_split avec une graine fixe pour la reproductibilité
train_subset, val_subset = random_split(
    trainset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(0)
)

# Création des DataLoaders (à compléter avec hparams)
trainloader = DataLoader(
    train_subset,
    batch_size=hparams["batch_size"],
    shuffle=True,
    pin_memory=True
)

valloader = DataLoader(
    val_subset,
    batch_size=hparams["batch_size"],
```

Résultat après la commande `python train_tb.py`:

```
[compute] [job 2311] mboukraa@arcadia-slurm-node-4:~/TP-HPC/TP1 (main)
$ python train_tb.py
Logdir: runs/MLP/bs32_lr0.01_20260918-125153
Nombre total d'images : 50000
Images d'entraînement : 45000
Images de validation : 5000
[compute] [job 2311] mboukraa@arcadia-slurm-node-4:~/TP-HPC/TP1 (main)
$
```

Réponse à la question:

Inclure les hyperparamètres ainsi que la date et l'heure dans le nom du dossier de logs permet d'identifier facilement chaque expérience et de les différencier. Les hyperparamètres permettent de savoir avec quelle configuration le modèle a été entraîné, tandis que la date et l'heure permettent de distinguer plusieurs exécutions ayant une configuration similaire.

Cela facilite la comparaison des différents runs dans TensorBoard et évite d'écraser les résultats d'expériences précédentes.

5.b.

La partie du code ajoutée:

```
# =====  
# Calcul des métriques sur un DataLoader  
# =====  
  
@torch.no_grad()  
def epoch_metrics(loader, model, criterion, device):  
    model.eval()  
    loss_sum, correct, total = 0.0, 0, 0  
  
    for x, y in loader:  
        x = x.to(device, non_blocking=True)  
        y = y.to(device, non_blocking=True)  
  
        logits = model(x)  
        loss = criterion(logits, y)  
  
        loss_sum += loss.item() * y.size(0)  
  
        pred = logits.argmax(1)  
        correct += (pred == y).sum().item()  
        total += y.size(0)  
  
    loss_avg = loss_sum / total  
    acc = correct / total  
  
    return loss_avg, acc
```

5.d.

Le transfert vers mon PC a bien été fait:

```
(base) meria@Meriam:/mnt/c/Users/meria$ scp -r tsp-client:~/TP-HPC/TP1/runs  
./runs  
mboukraa@ssh3.imtbs-tsp.eu's password:  
mboukraa@157.159.104.128's password:  
events.out.tfevents.1789735913.arcadia-sl 100% 88 0.4KB/s 00:00  
(base) meria@Meriam:/mnt/c/Users/meria$ ls runs  
tsp  
(base) meria@Meriam:/mnt/c/Users/meria$
```

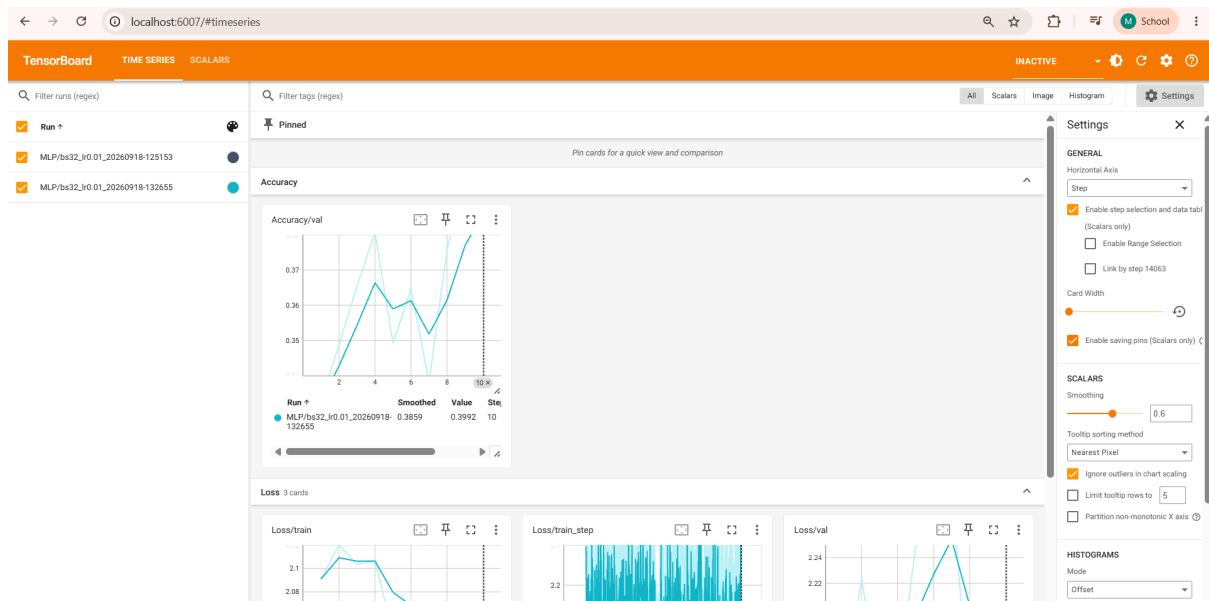
On peut donc lancer TensorBoard:

```
(base) meria@Meriam:/mnt/c/Users/meria$ tensorboard --logdir=./runs
TensorFlow installation not found - running with reduced feature set.

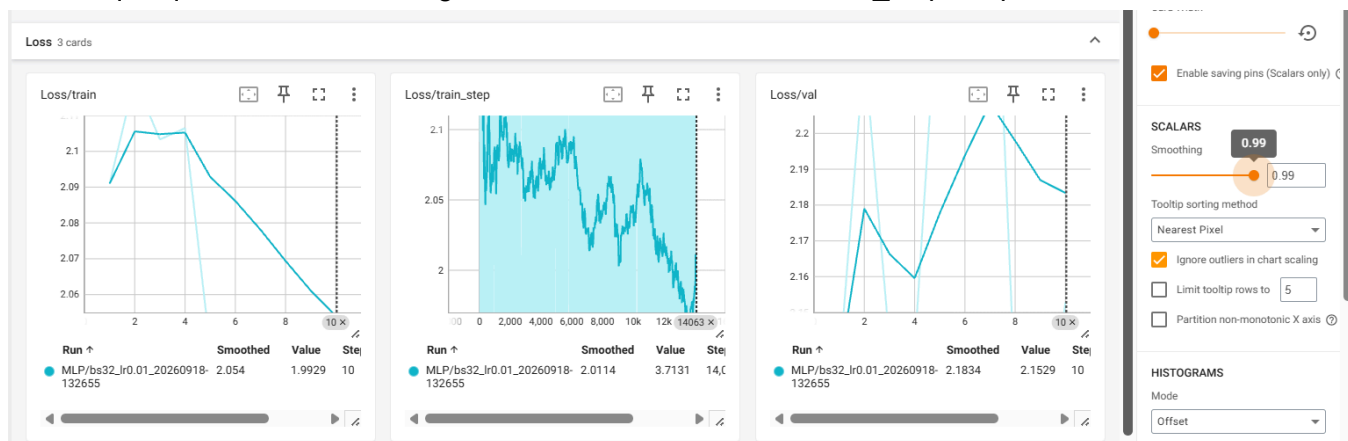
NOTE: Using experimental fast data loading logic. To disable, pass
"--load_fast=false" and report issues on GitHub. More details:
https://github.com/tensorflow/tensorboard/issues/4784

Serving TensorBoard on localhost; to expose to the network, use a proxy or pass
--bind_all
TensorBoard 2.21.0 at http://localhost:6006/ (Press CTRL+C to quit)
```

Voici le résultat dans l'interface de TensorBoard:



On remarque qu'avec un Smoothing de 0.99, la courbe de Loss/train_step est plus lisse:



La courbe Loss/train_step est plus bruitée que Loss/train car elle correspond à la loss calculée sur un seul mini-batch à chaque étape d'entraînement. Chaque mini-batch contient seulement 32 images et sa composition varie, ce qui entraîne des variations importantes de la loss d'un batch à l'autre.

À l'inverse, Loss/train correspond à la loss moyenne sur l'ensemble des 45 000 images d'entraînement pour une époque. Cette moyenne réduit naturellement les variations entre les mini-batches et produit donc une courbe beaucoup plus stable.

5.e.

- Run 2 : LR = $1e-3$, batch_size = 32:

```
# Hyperparamètres (faciles à modifier pour nos futures expériences)

hparams = dict(
    model="MLP",
    batch_size=32,
    lr=1e-3,
    seed=0,
    weight_decay=0.0
)
```

Résultat:

```
[compute] [job 2355] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP1 (main
)
$ python train_tb.py
Logdir: runs/MLP/bs32_lr0.001_20260918-140652
Nombre total d'images : 50000
Images d'entraînement : 45000
Images de validation : 5000
Using device: cuda
Epoch 01 | train_loss=1.6842 | val_loss=1.5935 | val_acc=0.441
Epoch 02 | train_loss=1.4866 | val_loss=1.5289 | val_acc=0.465
Epoch 03 | train_loss=1.3994 | val_loss=1.5071 | val_acc=0.472
Epoch 04 | train_loss=1.3372 | val_loss=1.4741 | val_acc=0.484
Epoch 05 | train_loss=1.2875 | val_loss=1.4585 | val_acc=0.491
Epoch 06 | train_loss=1.2440 | val_loss=1.4861 | val_acc=0.495
Epoch 07 | train_loss=1.2081 | val_loss=1.4808 | val_acc=0.484
Epoch 08 | train_loss=1.1724 | val_loss=1.4798 | val_acc=0.492
Epoch 09 | train_loss=1.1385 | val_loss=1.4780 | val_acc=0.498
Epoch 10 | train_loss=1.1077 | val_loss=1.4800 | val_acc=0.500
[compute] [job 2355] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP1 (main
)
```

- Run 3 : LR = $1e-1$, batch_size = 128:

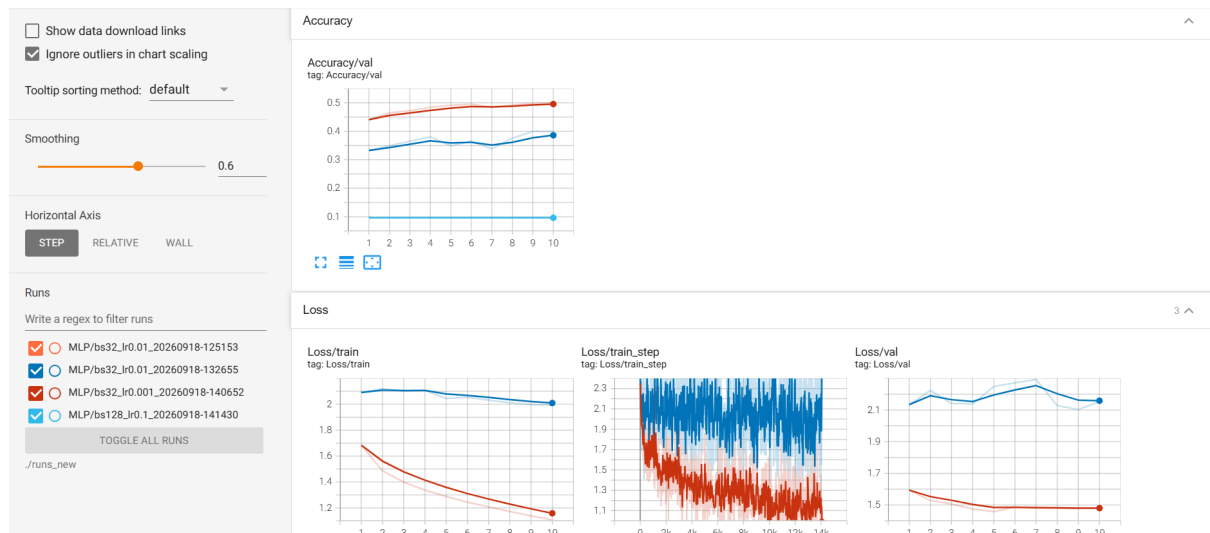
```
# Hyperparamètres (faciles à modifier pour nos futures expériences)

hparams = dict(
    model="MLP",
    batch_size=128,
    lr=1e-1,
    seed=0,
    weight_decay=0.0
)
```

Résultat:

```
[compute] [job 2358] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP1 (main)
$ python train_tb.py
Logdir: runs/MLP/bs128_lr0.1_20260918-141430
Nombre total d'images : 50000
Images d'entraînement : 45000
Images de validation : 5000
Using device: cuda
Epoch 01 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 02 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 03 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 04 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 05 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 06 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 07 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 08 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 09 | train_loss=nan | val_loss=nan | val_acc=0.096
Epoch 10 | train_loss=nan | val_loss=nan | val_acc=0.096
[compute] [job 2358] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP1 (main)
$
```

Réponse à la question 1:



Les trois runs présentent des comportements différents selon les hyperparamètres utilisés.

Pour le Run 1 (LR=1e-2, batch_size=32), la loss d'entraînement diminue globalement, mais l'accuracy de validation reste limitée et atteint 39,9 % à la fin des 10 époques.

Le Run 2 (LR=1e-3, batch_size=32) présente une diminution plus régulière de la Loss/train. La Loss/val diminue également au début, puis se stabilise autour de la cinquième époque. L'accuracy de validation atteint 50,0 %, ce qui est la valeur la plus élevée parmi les trois runs.

Pour le Run 3 (LR=1e-1, batch_size=128), les losses deviennent immédiatement NaN et l'accuracy de validation reste autour de 9,6 %. Cela indique que le learning rate est trop élevé pour ce modèle et provoque une instabilité de l'entraînement.

Ainsi, parmi les trois configurations testées, le Run 2 atteint l'accuracy de validation la plus élevée, avec 50,0 %.

Réponse à la question 2:

Un sur-apprentissage est identifiable lorsque la **Loss/train continue à diminuer**, tandis que la **Loss/val commence à augmenter**. Le modèle continue alors à améliorer ses performances sur les données d'entraînement, mais généralise moins bien sur les données de validation.

Dans notre cas, pour le Run 2, la Loss/train diminue régulièrement tandis que la Loss/val diminue également au début puis se stabilise autour de l'époque 5. On n'observe donc pas de divergence nette entre les deux courbes indiquant un sur-apprentissage important sur les 10 époques.